PYTHON PROGRAMMING

Activity-Based Learning (ABL) Assignment

Name: Deepika Bezawada **Roll Number:** 26000886

Branch/Section: CSM 1 **Course:** Python Programming

Selected 10 Case Studies

1. University Scholarship Allocation
2. Progressive Electricity Billing
3. Employee Payroll and Deduction Analysis
4. Retail Billing with Multiple Discount Rules
5. Attendance Recovery Computation
6. ATM Transaction Sequence Simulation
7. Warehouse Inventory Reconciliation
8. Student Marks Matrix Analysis
9. Password Policy Validation Engine
10. Campus Event Financial Planning

Note: These solutions use illustrative assumptions where the supplied case-study list does not specify exact rules or values. Review the programs, replace sample data if required, and ensure you can explain your work independently.

Case Study 1: University Scholarship Allocation

1. Problem Statement

Select scholarship applicants using aggregate marks, subject-wise minimum marks, attendance, and family income. Rank eligible applicants and resolve ties using aggregate marks, then attendance.

2. Objectives

• Validate applicant details and eligibility conditions.

• Calculate aggregate percentage from subject marks.

• Rank eligible applicants using clear tie-breaking rules.

3. Assumptions

• Each applicant has marks in three subjects, each out of 100.

• Eligibility requires aggregate percentage of at least 80%, every subject mark of at least 70, attendance of at least 85%, and annual family income no more than Rs. 300000.

• Higher aggregate ranks first; if tied, higher attendance ranks first.

4. Algorithm

Start and read the applicant records.

For each applicant, calculate aggregate percentage.

Check aggregate, subject marks, attendance, and income conditions.

Store eligible applicants in a list.

Sort eligible applicants by aggregate and attendance in descending order.

Display eligible rankings and rejected applicants with reasons.

Stop.

5. Flowchart

Start -> Initialize/read data -> Process calculations -> Check conditions/loop if required -> Display result -> Stop. Main decision point: Check aggregate, subject marks, attendance, and income conditions.

6. Pseudocode

```
READ applicant records
FOR each applicant
    percentage = sum(subject marks) / number of subjects
    IF percentage >= 80 AND all subject marks >= 70 AND attendance >= 85 AND income <= 300000
        add applicant to eligible list
    ELSE
        record reason for rejection
SORT eligible by percentage descending, then attendance descending
DISPLAY ranked eligible applicants and rejected applicants
```

7. Python Source Code

```
# Case Study 1: University Scholarship Allocation
applicants = [
    ("Asha", [88, 91, 84], 92, 250000),
    ("Bharat", [78, 90, 86], 90, 220000),
    ("Charan", [95, 88, 91], 82, 180000),
    ("Divya", [82, 85, 80], 88, 280000),
]

eligible = []
rejected = []

for name, marks, attendance, income in applicants:
    percentage = sum(marks) / len(marks)
    reasons = []
    if percentage < 80:
        reasons.append("aggregate below 80%")
    if min(marks) < 70:
        reasons.append("a subject mark below 70")
    if attendance < 85:
        reasons.append("attendance below 85%")
    if income > 300000:
        reasons.append("income above limit")

    if reasons:
        rejected.append((name, ", ".join(reasons)))
    else:
        eligible.append((name, percentage, attendance, income))

eligible.sort(key=lambda item: (item[1], item[2]), reverse=True)

print("SCHOLARSHIP RANK LIST")
for rank, item in enumerate(eligible, start=1):
    print(f"{rank}. {item[0]} - {item[1]:.2f}% aggregate, {item[2]}% attendance")
print("\nNOT ELIGIBLE")
for name, reason in rejected:
    print(f"{name}: {reason}")
```

8. Test Cases

Test	Input/condition	Expected result
Eligible applicant	Asha: 88,91,84; attendance 92; income 25000	Eligible; ranked by percentage
Low attendance	Charan attendance 82	Rejected: attendance below 85%
Aggregate below limit	Bharat marks 78,90,86	Rejected: aggregate below 80%

9. Sample Output

```
SCHOLARSHIP RANK LIST
1. Asha - 87.67% aggregate, 92% attendance
2. Bharat - 84.67% aggregate, 90% attendance
3. Divya - 82.33% aggregate, 88% attendance

NOT ELIGIBLE
Charan: attendance below 85%
```

10. Conclusion

The program applies multiple eligibility conditions and produces a ranked scholarship list with reasons for rejection.

Case Study 2: Progressive Electricity Billing

1. Problem Statement

Calculate a household electricity bill using consumption slabs, fixed charge, surcharge, and rebate. Compare bills from several months and flag a large increase.

2. Objectives

• Use conditional statements to calculate slab charges.

• Add fixed charges and surcharge, then apply a rebate.

• Compare monthly bills and identify unusual increases.

3. Assumptions

• First 100 units cost Rs. 2 per unit; next 100 units cost Rs. 3 per unit; units above 200 cost Rs. 5 per unit.

• Fixed charge is Rs. 50. Surcharge is 5% of energy charge. If consumption is below 100 units, rebate is Rs. 20.

• An increase above 50% compared with the previous month is unusual.

4. Algorithm

Read monthly unit readings.

Calculate energy charge using progressive slabs.

Add fixed charge and surcharge; apply rebate where eligible.

Store each total bill.

Compare each month with the previous month and flag increases above 50%.

Display bill details.

5. Flowchart

Start -> Initialize/read data -> Process calculations -> Check conditions/loop if required -> Display result -> Stop. Main decision point: Add fixed charge and surcharge; apply rebate where eligible.

6. Pseudocode

```
FOR each month's units
    IF units <= 100: energy = units * 2
    ELSE IF units <= 200: energy = 100*2 + (units-100)*3
    ELSE: energy = 100*2 + 100*3 + (units-200)*5
    bill = energy + 50 + energy*0.05
    IF units < 100: bill = bill - 20
COMPARE each bill with previous bill
FLAG increase > 50%
```

7. Python Source Code

```
# Case Study 2: Progressive Electricity Billing
monthly_units = [85, 120, 190, 310]
bills = []

for month, units in enumerate(monthly_units, start=1):
    if units <= 100:
        energy = units * 2
    elif units <= 200:
        energy = 100 * 2 + (units - 100) * 3
    else:
        energy = 100 * 2 + 100 * 3 + (units - 200) * 5

    bill = energy + 50 + energy * 0.05
    if units < 100:
        bill -= 20
    bills.append(bill)
    print(f"Month {month}: {units} units, energy Rs. {energy:.2f}, bill Rs. {bill:.2f}")

print("\nMONTH-TO-MONTH CHECK")
for i in range(1, len(bills)):
    increase = (bills[i] - bills[i - 1]) / bills[i - 1] * 100
    if increase > 50:
        print(f"Month {i + 1}: unusual increase of {increase:.1f}%")
    else:
        print(f"Month {i + 1}: increase of {increase:.1f}%")
```

8. Test Cases

Test	Input/condition	Expected result
Low use	85 units	Rebate applied
Middle slab	120 units	First 100 at Rs. 2, remaining at Rs. 3
High use	310 units	Units above 200 billed at Rs. 5

9. Sample Output

```
Month 1: 85 units, energy Rs. 170.00, bill Rs. 208.50
Month 2: 120 units, energy Rs. 260.00, bill Rs. 323.00
Month 3: 190 units, energy Rs. 470.00, bill Rs. 543.50
Month 4: 310 units, energy Rs. 1050.00, bill Rs. 1152.50
```

```
MONTH-TO-MONTH CHECK
Month 2: unusual increase of 54.9%
Month 3: unusual increase of 68.3%
Month 4: unusual increase of 112.1%
```

10. Conclusion

The program calculates progressive bills and identifies months with unusually high increases.

Case Study 3: Employee Payroll and Deduction Analysis

1. Problem Statement

Calculate employee gross salary from basic pay, allowances, overtime, and incentives. Apply progressive deductions and print a salary statement.

2. Objectives

• Calculate allowances and overtime pay.

• Apply deductions using salary ranges.

• Display gross pay, deductions, and net salary.

3. Assumptions

• HRA is 20% of basic pay and travel allowance is 10% of basic pay.

• Overtime is paid at Rs. 200 per hour; incentives are added directly.

• Deduction is 5% on the first Rs. 20000 of gross pay and 10% on the amount above Rs. 20000.

4. Algorithm

Read employee name, basic pay, overtime hours, and incentive.

Calculate HRA, travel allowance, and overtime pay.

Add these amounts to calculate gross pay.

Calculate progressive deduction.

Subtract deduction from gross pay.

Print salary statement.

5. Flowchart

Start -> Initialize/read data -> Process calculations -> Check conditions/loop if required -> Display result -> Stop. Main decision point: Add these amounts to calculate gross pay.

6. Pseudocode

```
READ name, basic, overtime_hours, incentive
hra = basic * 0.20
travel = basic * 0.10
overtime = overtime_hours * 200
gross = basic + hra + travel + overtime + incentive
first = min(gross, 20000) * 0.05
extra = max(gross - 20000, 0) * 0.10
deduction = first + extra
net = gross - deduction
DISPLAY salary statement
```

7. Python Source Code

```
# Case Study 3: Employee Payroll
name = "Ravi"
basic = 24000
overtime_hours = 6
incentive = 1500

hra = basic * 0.20
travel_allowance = basic * 0.10
overtime_pay = overtime_hours * 200
gross = basic + hra + travel_allowance + overtime_pay + incentive

deduction_first = min(gross, 20000) * 0.05
deduction_extra = max(gross - 20000, 0) * 0.10
deduction = deduction_first + deduction_extra
net_salary = gross - deduction

print("EMPLOYEE SALARY STATEMENT")
print("Employee:", name)
print(f"Basic pay: Rs. {basic:.2f}")
print(f"HRA: Rs. {hra:.2f}")
print(f"Travel allowance: Rs. {travel_allowance:.2f}")
print(f"Overtime pay: Rs. {overtime_pay:.2f}")
print(f"Incentive: Rs. {incentive:.2f}")
print(f"Gross salary: Rs. {gross:.2f}")
print(f"Deduction: Rs. {deduction:.2f}")
print(f"Net salary: Rs. {net_salary:.2f}")
```

8. Test Cases

Test	Input/condition	Expected result
Overtime	6 hours	Overtime pay is Rs. 1200
No overtime	0 hours	Overtime pay is Rs. 0
Higher gross	Gross above Rs. 20000	Extra portion receives 10% deduction

9. Sample Output

```
EMPLOYEE SALARY STATEMENT
```

```
Employee: Ravi
Basic pay: Rs. 24000.00
HRA: Rs. 4800.00
Travel allowance: Rs. 2400.00
Overtime pay: Rs. 1200.00
Incentive: Rs. 1500.00
Gross salary: Rs. 33900.00
Deduction: Rs. 2390.00
Net salary: Rs. 31510.00
```

10. Conclusion

The program creates a readable salary statement and demonstrates conditional progressive deductions.

Case Study 4: Retail Billing with Multiple Discount Rules

1. Problem Statement

Generate an itemized retail invoice for several products. Apply a discount when the subtotal reaches a threshold, category-based tax, and a maximum discount limit.

2. Objectives

• Store product details in lists and tuples.

• Calculate subtotal, discount, tax, and final amount.

• Print an itemized bill.

3. Assumptions

• Products are represented by (name, price, quantity, tax rate) tuples.

• A 10% discount applies if subtotal is at least Rs. 2000, capped at Rs. 300.

• Tax is calculated on the discounted subtotal.

4. Algorithm

Store product tuples in a list.

Calculate each line total and the subtotal.

Calculate discount and apply the maximum cap.

Calculate tax on the discounted subtotal using each product's tax rate.

Display line totals and invoice total.

5. Flowchart

Start -> Initialize/read data -> Process calculations -> Check conditions/loop if required -> Display result -> Stop. Main decision point: Calculate discount and apply the maximum cap.

6. Pseudocode

```
subtotal = 0
FOR each product tuple
    line_total = price * quantity
    subtotal = subtotal + line_total
IF subtotal >= 2000: discount = min(subtotal * 0.10, 300)
ELSE: discount = 0
FOR each product
    taxable_line = line_total - proportional discount
    tax = taxable_line * tax_rate
final_total = subtotal - discount + total_tax
DISPLAY invoice
```

7. Python Source Code

```
# Case Study 4: Retail Billing
products = [
    ("Notebook", 80, 5, 0.05),
    ("School Bag", 900, 2, 0.12),
    ("Pen Pack", 120, 3, 0.05),
]

subtotal = 0
for name, price, quantity, tax_rate in products:
    subtotal += price * quantity

discount = min(subtotal * 0.10, 300) if subtotal >= 2000 else 0
discount_ratio = discount / subtotal if subtotal else 0
total_tax = 0

print("ITEMIZED INVOICE")
for name, price, quantity, tax_rate in products:
    line_total = price * quantity
    line_discount = line_total * discount_ratio
    taxable_amount = line_total - line_discount
    tax = taxable_amount * tax_rate
    total_tax += tax
    print(f"{name}: {quantity} x Rs. {price} = Rs. {line_total:.2f}; tax Rs. {tax:.2f}")

final_total = subtotal - discount + total_tax
print(f"Subtotal: Rs. {subtotal:.2f}")
print(f"Discount: Rs. {discount:.2f}")
print(f"Tax: Rs. {total_tax:.2f}")
print(f"FINAL AMOUNT: Rs. {final_total:.2f}")
```

8. Test Cases

Test	Input/condition	Expected result
Subtotal below threshold	Subtotal less than Rs. 2000	No discount
Subtotal above threshold	Subtotal Rs. 2000 or more	10% discount, maximum Rs. 300
Different tax rates	Products use 5% and 12%	Tax calculated per product

9. Sample Output

```
ITEMIZED INVOICE
Notebook: 5 x Rs. 80 = Rs. 400.00; tax Rs. 18.00
School Bag: 2 x Rs. 900 = Rs. 1800.00; tax Rs. 194.40
Pen Pack: 3 x Rs. 120 = Rs. 360.00; tax Rs. 16.20
Subtotal: Rs. 2560.00
Discount: Rs. 256.00
Tax: Rs. 228.60
FINAL AMOUNT: Rs. 2532.60
```

10. Conclusion

The program uses tuples, loops, and arithmetic to generate a clear invoice with discounts and different tax rates.

Case Study 5: Attendance Recovery Computation

1. Problem Statement

Calculate current attendance and determine the minimum number of consecutive classes a student must attend to reach a required attendance percentage.

2. Objectives

• Calculate attendance percentage.

• Use a loop to find the minimum classes required.

• Handle students who already meet the threshold.

3. Assumptions

• Attendance eligibility threshold is 75%.

• A future attended class increases both classes attended and total classes by one.

• The program uses a maximum search limit of 10000 classes to avoid an endless loop.

4. Algorithm

Read classes attended and total classes.

Validate that total is positive and attended is between zero and total.

Calculate current percentage.

If below threshold, repeatedly add one attended class and one total class until threshold is reached.

Display current percentage and required classes.

5. Flowchart

Start -> Initialize/read data -> Process calculations -> Check conditions/loop if required -> Display result -> Stop. Main decision point: Calculate current percentage.

6. Pseudocode

```
READ attended, total
IF total <= 0 OR attended < 0 OR attended > total: DISPLAY invalid input
ELSE
    current = attended / total * 100
    required = 0
    WHILE (attended + required)/(total + required)*100 < 75
        required = required + 1
    DISPLAY current and required
```

7. Python Source Code

```
# Case Study 5: Attendance Recovery
attended = 42
total_classes = 60
threshold = 75

if total_classes <= 0 or attended < 0 or attended > total_classes:
    print("Invalid attendance data")
else:
    current = attended / total_classes * 100
    required = 0

    while (attended + required) / (total_classes + required) * 100 < threshold:
        required += 1
        if required > 10000:
            break

    print(f"Classes attended: {attended}")
    print(f"Total classes: {total_classes}")
    print(f"Current attendance: {current:.2f}%")
    if current >= threshold:
        print("You already meet the attendance requirement.")
    else:
        print(f"Attend the next {required} consecutive classes to reach {threshold}%.")
        final_percent = (attended + required) / (total_classes + required) * 100
        print(f"Attendance after recovery: {final_percent:.2f}%")
```

8. Test Cases

Test	Input/condition	Expected result
Below threshold	42/60 classes	Needs 12 consecutive classes to reach 75%
Already eligible	48/60 classes	No extra classes required
Invalid data	attended greater than total	Invalid attendance data

9. Sample Output

```
Classes attended: 42
Total classes: 60
Current attendance: 70.00%
Attend the next 12 consecutive classes to reach 75%.
Attendance after recovery: 75.00%
```

10. Conclusion

The program calculates attendance and uses a loop to find the smallest number of future classes needed.

Case Study 6: ATM Transaction Sequence Simulation

1. Problem Statement

Simulate a series of deposits and withdrawals, maintain an account balance, reject invalid transactions, enforce a withdrawal limit, and summarize accepted transactions.

2. Objectives

• Process transactions using tuples and loops.

• Validate amounts and available balance.

• Summarize deposits, withdrawals, and rejected transactions.

3. Assumptions

• Opening balance is Rs. 5000.

• Each withdrawal must be no more than Rs. 3000 and no more than the available balance.

• Transactions are tuples containing type and amount; valid types are DEPOSIT and WITHDRAW.

4. Algorithm

Set opening balance and transaction list.

For each transaction, check type and amount.

Accept deposits with positive amounts.

Accept withdrawals only if amount is positive, within limit, and balance is sufficient.

Update balance and totals, or record rejection.

Display final balance and transaction summary.

5. Flowchart

Start -> Initialize/read data -> Process calculations -> Check conditions/loop if required -> Display result -> Stop. Main decision point: Accept deposits with positive amounts.

6. Pseudocode

```
balance = opening balance
FOR each (type, amount)
    IF amount <= 0: reject
    ELSE IF type is DEPOSIT: add amount to balance
    ELSE IF type is WITHDRAW AND amount <= 3000 AND amount <= balance: subtract amount
    ELSE: reject
DISPLAY final balance, deposits, withdrawals, rejected count
```

7. Python Source Code

```
# Case Study 6: ATM Transaction Sequence
transactions = [
    ("DEPOSIT", 2000),
    ("WITHDRAW", 1500),
    ("WITHDRAW", 4000),
    ("DEPOSIT", 0),
    ("WITHDRAW", 1000),
]
balance = 5000
total_deposits = 0
total_withdrawals = 0
rejected = 0
withdrawal_limit = 3000

for transaction_type, amount in transactions:
    if amount <= 0:
        print(f"Rejected: {transaction_type} Rs. {amount} (amount must be positive)")
        rejected += 1
    elif transaction_type == "DEPOSIT":
        balance += amount
        total_deposits += amount
        print(f"Accepted deposit: Rs. {amount}")
    elif transaction_type == "WITHDRAW":
        if amount > withdrawal_limit or amount > balance:
            print(f"Rejected withdrawal: Rs. {amount} (limit or balance exceeded)")
            rejected += 1
        else:
            balance -= amount
            total_withdrawals += amount
            print(f"Accepted withdrawal: Rs. {amount}")
    else:
        print("Rejected: unknown transaction type")
        rejected += 1

print("\nATM SUMMARY")
print(f"Deposits: Rs. {total_deposits}")
print(f"Withdrawals: Rs. {total_withdrawals}")
print(f"Rejected transactions: {rejected}")
print(f"Closing balance: Rs. {balance}")
```

8. Test Cases

Test	Input/condition	Expected result
Valid deposit	Deposit Rs. 2000	Balance increases by Rs. 2000
Over-limit withdrawal	Withdraw Rs. 4000	Rejected because limit is Rs. 3000
Zero amount	Deposit Rs. 0	Rejected

9. Sample Output

```
Accepted deposit: Rs. 2000
Accepted withdrawal: Rs. 1500
Rejected withdrawal: Rs. 4000 (limit or balance exceeded)
Rejected: DEPOSIT Rs. 0 (amount must be positive)
Accepted withdrawal: Rs. 1000

ATM SUMMARY
Deposits: Rs. 2000
Withdrawals: Rs. 2500
Rejected transactions: 2
Closing balance: Rs. 4500
```

10. Conclusion

The simulation demonstrates tuple unpacking, validation, conditional logic, and transaction summaries.

Case Study 7: Warehouse Inventory Reconciliation

1. Problem Statement

Compare opening stock, incoming stock, dispatched stock, and recorded closing stock for products. Detect differences and calculate replenishment needs.

2. Objectives

• Represent inventory records using tuples.

• Calculate expected closing stock.

• Identify discrepancies and reorder quantities.

3. Assumptions

• Expected closing stock = opening + incoming - dispatched.

• Reorder target is 20 units per product.

• A discrepancy exists if expected and recorded closing stock differ.

4. Algorithm

Store each product as a tuple.

Calculate expected closing stock.

Compare expected with recorded closing stock.

Calculate replenishment needed to reach target stock of 20.

Display discrepancy and replenishment results.

5. Flowchart

Start -> Initialize/read data -> Process calculations -> Check conditions/loop if required -> Display result -> Stop. Main decision point: Compare expected with recorded closing stock.

6. Pseudocode

```
FOR each product tuple
    expected = opening + incoming - dispatched
    difference = recorded_closing - expected
    IF difference != 0: flag discrepancy
    reorder = max(20 - recorded_closing, 0)
    DISPLAY product, expected, difference, reorder
```

7. Python Source Code

```
# Case Study 7: Warehouse Inventory Reconciliation
# Tuple: (product, opening, incoming, dispatched, recorded_closing)
records = [
    ("Pens", 50, 30, 45, 35),
    ("Notebooks", 80, 20, 70, 25),
    ("Folders", 40, 10, 25, 28),
]
target_stock = 20

print("INVENTORY RECONCILIATION")
for product, opening, incoming, dispatched, closing in records:
    expected = opening + incoming - dispatched
    difference = closing - expected
    reorder = max(target_stock - closing, 0)

    print(f"\nProduct: {product}")
    print(f"Expected closing stock: {expected}")
    print(f"Recorded closing stock: {closing}")
    if difference == 0:
        print("Status: Matched")
    else:
        print(f"Status: DISCREPANCY ({difference:+d} units)")
    print(f"Replenishment required: {reorder} units")
```

8. Test Cases

Test	Input/condition	Expected result
Matching record	Opening 50, incoming 30, dispatched 45, closing 35	No discrepancy
Shortage	Expected 30, recorded 25	Discrepancy of -5
Above target	Recorded stock 35	No replenishment required

9. Sample Output

```
INVENTORY RECONCILIATION

Product: Pens
Expected closing stock: 35
Recorded closing stock: 35
Status: Matched
Replenishment required: 0 units

Product: Notebooks
Expected closing stock: 30
```

```
Recorded closing stock: 25
Status: DISCREPANCY (-5 units)
Replenishment required: 0 units

Product: Folders
Expected closing stock: 25
Recorded closing stock: 28
Status: DISCREPANCY (+3 units)
Replenishment required: 0 units
```

10. Conclusion

The program reconciles stock records and identifies discrepancies and replenishment needs.

Case Study 8: Student Marks Matrix Analysis

1. Problem Statement

Analyse a nested list of student marks. Calculate each student's total and average, each subject's average, identify the top student, and flag subjects with low averages.

2. Objectives

• Use nested lists to store a marks matrix.

• Calculate row-wise and column-wise statistics.

• Identify top performer and subjects needing attention.

3. Assumptions

• Each row represents one student and each column represents a subject.

• All marks are out of 100.

• A subject average below 50 is flagged as low.

4. Algorithm

Store student names, subject names, and marks matrix.

For each row, calculate total and average.

For each subject column, calculate average.

Identify the student with the highest average.

Flag subjects with averages below 50.

5. Flowchart

Start -> Initialize/read data -> Process calculations -> Check conditions/loop if required -> Display result -> Stop. Main decision point: For each subject column, calculate average.

6. Pseudocode

```
FOR each student row
    total = sum(row)
    average = total / number of subjects
FOR each subject column
    subject_average = sum(column marks) / number of students
FIND student with maximum average
FLAG subject average below 50
DISPLAY all results
```

7. Python Source Code

```
# Case Study 8: Student Marks Matrix
students = ["Asha", "Bharat", "Charan", "Divya"]
subjects = ["Python", "Maths", "English"]
marks = [
    [88, 76, 82],
    [72, 68, 75],
    [45, 52, 48],
    [91, 89, 94],
]

averages = []
print("STUDENT RESULTS")
for i in range(len(students)):
    total = sum(marks[i])
    average = total / len(subjects)
    averages.append(average)
    print(f"{students[i]}: total={total}, average={average:.2f}")

print("\nSUBJECT AVERAGES")
for j in range(len(subjects)):
    subject_total = 0
    for i in range(len(students)):
        subject_total += marks[i][j]
    subject_average = subject_total / len(students)
    print(f"{subjects[j]}: {subject_average:.2f}")
    if subject_average < 50:
        print("  Warning: low subject average")

top_index = averages.index(max(averages))
print(f"\nTop performer: {students[top_index]} ({averages[top_index]:.2f} average)")
```

8. Test Cases

Test	Input/condition	Expected result
Student average	Asha marks 88,76,82	Average is 82.00
Top performer	Divya marks 91,89,94	Divya is top performer
Low subject average	Data includes lower Python marks	Program flags subject average below 50 if threshold met

9. Sample Output

```
STUDENT RESULTS
```

```
Asha: total=246, average=82.00
Bharat: total=215, average=71.67
Charan: total=145, average=48.33
Divya: total=274, average=91.33

SUBJECT AVERAGES
Python: 74.00
Maths: 71.25
English: 74.75

Top performer: Divya (91.33 average)
```

10. Conclusion

The program uses nested lists and loops to calculate student-wise and subject-wise performance.

Case Study 9: Password Policy Validation Engine

1. Problem Statement

Validate sample passwords against length, uppercase, lowercase, digit, special-character, repeated-character, and prohibited-substring rules. Report all failed rules.

2. Objectives

• Use string operations to inspect passwords.

• Apply multiple validation conditions.

• Explain why a password fails.

3. Assumptions

• A valid password has at least 8 characters, one uppercase, one lowercase, one digit, one special character, no three identical consecutive characters, and does not contain 'password' ignoring case.

• This is an educational demonstration, not a real authentication system.

4. Algorithm

Read each sample password.

Check length and character categories.

Check for three identical consecutive characters.

Check prohibited substring case-insensitively.

Display valid/invalid status and each failure.

5. Flowchart

Start -> Initialize/read data -> Process calculations -> Check conditions/loop if required -> Display result -> Stop. Main decision point: Check for three identical consecutive characters.

6. Pseudocode

```
FOR each password
    failures = empty list
    CHECK length >= 8
    CHECK uppercase, lowercase, digit, special character
    CHECK no three identical consecutive characters
    CHECK 'password' not in lowercase password
    IF failures empty: valid
    ELSE: display failures
```

7. Python Source Code

```
# Case Study 9: Password Policy Validation
passwords = ["Hello@123", "abc123", "AAAhello1!", "MyPassword9!"]

for password in passwords:
    failures = []
    if len(password) < 8:
        failures.append("must contain at least 8 characters")
    if not any(ch.isupper() for ch in password):
        failures.append("needs an uppercase letter")
    if not any(ch.islower() for ch in password):
        failures.append("needs a lowercase letter")
    if not any(ch.isdigit() for ch in password):
        failures.append("needs a digit")
    if not any(not ch.isalnum() for ch in password):
        failures.append("needs a special character")

    repeated = False
    for i in range(len(password) - 2):
        if password[i] == password[i + 1] == password[i + 2]:
            repeated = True
    if repeated:
        failures.append("must not contain three identical consecutive characters")
    if "password" in password.lower():
        failures.append("contains prohibited word 'password'")

    print(f"\nPassword sample: {password}")
    if failures:
        print("INVALID")
        for failure in failures:
            print("-", failure)
    else:
        print("VALID: all policy checks passed")
```

8. Test Cases

Test	Input/condition	Expected result
Strong sample	Hello@123	Valid
Too short	abc123	Fails length and character-category checks
Repeated characters	AAAhello1!	Fails repeated-character rule
Prohibited word	MyPassword9!	Fails prohibited-substring rule

9. Sample Output

```
Password sample: Hello@123
VALID: all policy checks passed

Password sample: abc123
INVALID
- must contain at least 8 characters
- needs an uppercase letter
- needs a special character

Password sample: AAAhello1!
INVALID
- must not contain three identical consecutive characters

Password sample: MyPassword9!
INVALID
- contains prohibited word 'password'
```

10. Conclusion

The program checks password samples against multiple rules and reports specific failures. Real systems should use secure password handling.

Case Study 10: Campus Event Financial Planning

1. Problem Statement

Calculate the total cost of a campus event, expected sponsorship, participant contributions, sponsorship shortfall, and minimum number of registrations needed to break even.

2. Objectives

• Add fixed and per-participant expenses.

• Calculate funds available and funding gap.

• Use arithmetic and conditional logic to estimate break-even registration.

3. Assumptions

• Venue Rs. 12000, equipment Rs. 8000, transport Rs. 5000, certificates Rs. 30 per participant, catering Rs. 150 per participant.

• Sponsorship is Rs. 18000 and registration fee is Rs. 250 per participant.

• Break-even registrations are the smallest whole number for which sponsorship plus fees covers total cost.

4. Algorithm

Read expected participants and cost details.

Calculate fixed costs and variable cost per participant.

Calculate total event cost and available funds.

Calculate funding surplus or shortfall.

Loop from zero registrations until funds cover total cost.

Display budget summary and break-even registrations.

5. Flowchart

Start -> Initialize/read data -> Process calculations -> Check conditions/loop if required -> Display result -> Stop. Main decision point: Calculate total event cost and available funds.

6. Pseudocode

```
fixed = venue + equipment + transport
variable_per_person = certificate_cost + catering_cost
total_cost = fixed + participants * variable_per_person
funds = sponsorship + participants * registration_fee
shortfall = max(total_cost - funds, 0)
registrations = 0
WHILE sponsorship + registrations*fee < fixed + registrations*variable_per_person
    registrations = registrations + 1
DISPLAY cost, funds, shortfall, break-even registrations
```

7. Python Source Code

```
# Case Study 10: Campus Event Financial Planning
venue = 12000
equipment = 8000
transport = 5000
certificate_per_person = 30
catering_per_person = 150
sponsorship = 18000
registration_fee = 250
expected_participants = 100

fixed_cost = venue + equipment + transport
variable_cost = certificate_per_person + catering_per_person
total_cost = fixed_cost + expected_participants * variable_cost
funds = sponsorship + expected_participants * registration_fee
shortfall = max(total_cost - funds, 0)

break_even = 0
while sponsorship + break_even * registration_fee < fixed_cost + break_even * variable_cost:
    break_even += 1

print("CAMPUS EVENT BUDGET")
print(f"Fixed costs: Rs. {fixed_cost}")
print(f"Variable cost per participant: Rs. {variable_cost}")
print(f"Expected participants: {expected_participants}")
print(f"Total expected cost: Rs. {total_cost}")
print(f"Sponsorship: Rs. {sponsorship}")
print(f"Registration contributions: Rs. {expected_participants * registration_fee}")
print(f"Total funds: Rs. {funds}")
if shortfall:
    print(f"Funding shortfall: Rs. {shortfall}")
else:
    print(f"Funding surplus: Rs. {funds - total_cost}")
print(f"Break-even registrations with current sponsorship: {break_even}")
```

8. Test Cases

Test	Input/condition	Expected result
Expected 100 participants	100 participants	Calculate total cost and funding gap
Sponsorship covers costs	Increase sponsorship	Shortfall becomes zero if funds are sufficient

Test	Input/condition	Expected result
Break-even	Fee Rs. 250, variable cost Rs. 180	Each registration contributes Rs. 70 toward fixed costs

9. Sample Output

```
CAMPUS EVENT BUDGET
Fixed costs: Rs. 25000
Variable cost per participant: Rs. 180
Expected participants: 100
Total expected cost: Rs. 43000
Sponsorship: Rs. 18000
Registration contributions: Rs. 25000
Total funds: Rs. 43000
Funding surplus: Rs. 0
Break-even registrations with current sponsorship: 100
```

10. Conclusion

The program calculates event finances and determines the minimum registration count needed to break even under the stated assumptions.